Assignment 4: Word Embeddings + Emotion Recognition

(Cheating/plagrism or ai generated code/assignments leads to rejection and serious actions towards certificate generation)

Title

Deep Dive into Word Embeddings: Training Word2Vec and Emotion Classification using GloVe

Problem Statement

Master word embeddings by training custom Word2Vec models on domain-specific corpora and utilizing pre-trained GloVe embeddings for emotion recognition. You will explore embedding properties through visualization and analogy tasks, then build a multi-class emotion classifier that demonstrates the superiority of dense embeddings over traditional sparse representations.

Dataset

Primary: GoEmotions Dataset

- **Source**: Google GoEmotions or Kaggle GoEmotions
- **Description**: 58,000 Reddit comments labeled with 27 emotion categories + neutral
- **Emotions**: Admiration, amusement, anger, annoyance, approval, caring, confusion, curiosity, desire, disappointment, disapproval, disgust, embarrassment, excitement, fear, gratitude, grief, joy, love, nervousness, optimism, pride, realization, relief, remorse, sadness, surprise

For Word2Vec Training: Domain-Specific Corpus

- **Option 1**: Medical texts from PubMed
- **Option 2**: Legal documents from Caselaw Access Project
- **Option 3**: News articles from News Category Dataset
- **Minimum size**: 10MB text, 1 million words

Detailed Instructions

Part 1: Word2Vec from Scratch (30%)

Task 1.1: Data Preparation for Word2Vec

1. Download and load domain-specific corpus (10,000+ documents)
2. Preprocess text:

- Lowercase conversion
- Remove special characters (keep hyphens in words)
- Sentence tokenization
- Word tokenization

3. Create vocabulary:
   - Filter words appearing < 5 times
   - Build word-to-index and index-to-word mappings
4. Generate training data for Skip-gram or CBOW architecture

Task 1.2: Train Custom Word2Vec Model Using gensim library:

Python code template:

```
from gensim.models import Word2Vec
# Prepare sentences
sentences = [['word1', 'word2', ...], ...]
# Train model
model = Word2Vec(
  sentences=sentences,
  vector_size=100,    # Embedding dimension
  window=5,          # Context window
  min_count=5,       # Ignore rare words
  workers=4,         # Parallel processing
  sg=1,              # Skip-gram (1) or CBOW (0)
  epochs=10,
  negative=5         # Negative sampling
)
# Save model
model.save("word2vec_custom.model")
```

Task 1.3: Experiment with Hyperparameters Train multiple models with different configurations:

1. **Vector size**: 50, 100, 200, 300
2. **Window size**: 2, 5, 10
3. **Architecture**: Skip-gram vs CBOW
4. **Training epochs**: 5, 10, 20
5. Document impact on embedding quality

Task 1.4: Embedding Analysis

1. **Similarity Tasks**:

   python code implementation

   ```
   # Most similar words
   model.wv.most_similar('doctor', topn=10)
   # Similarity score
   model.wv.similarity('hospital', 'clinic')
   ```

2. **Analogy Tasks**:

   Python code implementation example

   ```
   # king - man + woman = ?
   model.wv.most_similar(
     positive=['king', 'woman'],
     negative=['man'],
     topn=1
   )
   ```

   Test on domain-specific analogies

3. **Visualization**:
   - Use t-SNE to reduce embeddings to 2D
   - Plot 500 most frequent words
   - Color-code by POS tags or semantic categories
   - Identify clusters

4. **Intrinsic Evaluation**:
    - Evaluate on word similarity datasets (WordSim-353, SimLex-999)
    - Calculate Spearman correlation
    - Compare custom embeddings with pre-trained Word2Vec

Part 2: GloVe Embeddings for Emotion Recognition (40%)

Task 2.1: Load Pre-trained GloVe Embeddings

1. Download GloVe embeddings:
    - **Source**: Stanford GloVe
    - Use: glove.6B.100d.txt or glove.6B.300d.txt
2. Load embeddings into dictionary:

    Python code implementation example:

```
def load_glove(filepath):
    embeddings = {}
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            values = line.split()
            word = values[0]
            vector = np.array(values[1:], dtype='float32')
            embeddings[word] = vector
    return embeddings
```

3. Analyze GloVe vocabulary:
    - Total words
    - Coverage on your dataset
    - Handle OOV (Out-of-Vocabulary) words

Task 2.2: Prepare GoEmotions Dataset

1. Load dataset (58,000 comments with emotion labels)
2. Exploratory Data Analysis:
    - Class distribution (bar chart of 27 emotions)
    - Handle class imbalance (identify rare emotions)

- Analyze comment length distribution
- Sample visualization (5 examples per emotion)

3. Preprocessing:
    - Clean text (remove URLs, special chars)
    - Tokenization
    - Keep emotionally significant words (don't remove "not", "no")
4. Train/Val/Test split: 70/15/15 (stratified)

Task 2.3: Create Embedding Matrix

1. Build vocabulary from training data
2. Create embedding matrix:
3. Calculate embedding coverage percentage

Task 2.4: Build Neural Network Classifier

Model Architecture Options:

Option A: Simple Dense Network

python

Input (sequence_length) → Embedding Layer (pretrained GloVe, trainable=False) → GlobalAveragePooling1D → Dense(128, ReLU) → Dropout(0.5) → Dense(64, ReLU) → Dropout(0.3) → Dense (27, Softmax)

Option B: CNN for Text

Input → Embedding Layer → Conv1D(128, kernel_size=5) → GlobalMaxPooling1D → Dense(128, ReLU) → Dropout(0.5) → Dense(27, Softmax)

Option C: Bidirectional LSTM

Input → Embedding Layer → Bidirectional LSTM(64) → Dropout(0.5) → Dense(64, ReLU) → Dense(27, Softmax)

Implement at least 2 architectures and compare.

Task 2.5: Training Strategy

1. **Handle Class Imbalance**:
    - Calculate class weights
    - Use weighted loss function
    - Or use oversampling/undersampling

2. **Hyperparameters**:
    - Batch size: 32 or 64
    - Epochs: 20-50
    - Optimizer: Adam (lr=0.001)
    - Loss: Categorical cross-entropy
3. **Regularization**:
    - Dropout layers
    - L2 regularization
    - Early stopping (patience=5)
4. **Experiments**:
    - Frozen embeddings vs fine-tuned embeddings
    - Different embedding dimensions (100d vs 300d)
    - Different sequence lengths (50, 100, 200)

Task 2.6: Advanced: Multi-label Classification GoEmotions allows multiple emotions per comment:

1. Convert to multi-label format (binary matrix)
2. Use sigmoid activation instead of softmax
3. Use binary_crossentropy loss
4. Evaluate with:
    - Hamming loss
    - F1-score (micro, macro, weighted)
    - Precision@k, Recall@k

Part 3: Baseline Comparison (10%)

Task 3.1: Implement TF-IDF + ML Baseline

1. Train Logistic Regression with TF-IDF features
2. Train Random Forest with TF-IDF
3. Compare with GloVe-based models

Task 3.2: Compare Word2Vec vs GloVe

1. Use your custom Word2Vec embeddings for emotion classification
2. Compare with GloVe embeddings
3. Analyze which performs better and why

Part 4: Embedding Visualization & Analysis (15%)

Task 4.1: t-SNE Visualization

1. Extract embeddings for emotion-related words:
    - Happy words: ["joy", "happy", "excited", "thrilled"]
    - Sad words: ["sad", "depressed", "unhappy", "miserable"]
    - Anger words: ["angry", "furious", "mad", "rage"]
2. Visualize using t-SNE (2D and 3D)
3. Color-code by emotion category
4. Observe clustering patterns

Task 4.2: Embedding Arithmetic

1. Test emotion-related analogies:
    - happy - joy + sadness = ?
    - love - like + hate = ?
2. Explore semantic relationships:
    - Emotion intensity: "happy" → "ecstatic"
    - Emotion opposition: "joy" ↔ "sadness"

Task 4.3: Attention Visualization (if using attention mechanism)

1. Visualize attention weights for sample predictions
2. Identify which words contribute most to emotion classification
3. Create heatmap overlays on text

Task 4.4: Embedding Space Analysis

1. Calculate average embedding for each emotion class
2. Visualize emotion centroids in 2D space
3. Measure inter-class distances
4. Create emotion similarity matrix

Part 5: Error Analysis (5%)

1. Generate confusion matrix (27×27)

2. Identify most confused emotion pairs
3. Analyze misclassified examples:
    - Extract 10 examples for each major error pattern
    - Hypothesize reasons for misclassification
4. Analyze performance by:
    - Comment length
    - Emotion frequency (rare vs common)
    - Lexical complexity

Expected Outputs:

1. **Code Deliverables**
    - word2vec_training.py - Custom Word2Vec training
    - glove_loader.py - GloVe loading utilities
    - emotion_classifier.py - Neural network models
    - embedding_analysis.py - Visualization and analysis
    - Jupyter notebook with complete workflow
    - Requirements.txt
2. **Model Files**
    - Custom Word2Vec model (.model)
    - Trained emotion classifiers (.h5 or .pt)
    - Embedding matrices (NumPy arrays)
    - Vocabulary mappings (JSON)
3. **Visualizations** (minimum 15)
    - Word2Vec parameter comparison (accuracy vs vector size, window size)
    - t-SNE visualization of custom Word2Vec (2D scatter)
    - GloVe embedding space (t-SNE with emotion words highlighted)
    - GoEmotions class distribution (bar chart)
    - Training/validation loss curves
    - Training/validation accuracy curves
    - Confusion matrix (heatmap)
    - Per-class F1-scores (horizontal bar chart)

- Embedding coverage analysis
- Model comparison (TF-IDF vs Word2Vec vs GloVe)
- Attention heatmaps (if applicable)
- Emotion centroid visualization
- Emotion similarity matrix
- Error analysis by comment length
- Most confused emotion pairs

4. **Technical Report** (6-8 pages)
    - **Section 1: Word2Vec Training**
        - Corpus description and preprocessing
        - Hyperparameter experiments and results
        - Intrinsic evaluation (analogy tasks, similarity)
        - Visualization and insights
    - **Section 2: Emotion Recognition**
        - Dataset analysis and preprocessing
        - Model architecture(s) description
        - Training strategy and regularization
        - Results and comparison with baselines
    - **Section 3: Embedding Analysis**
        - Word2Vec vs GloVe comparison
        - Embedding visualization insights
        - Semantic properties discovered
    - **Section 4: Error Analysis**
        - Confusion patterns
        - Failure case analysis
        - Limitations and improvements
    - **Conclusion**: Key findings and recommendations

Findings/Observations to Document:

1. How does Word2Vec window size affect semantic vs syntactic relationships?
2. What's the trade-off between Skip-gram and CBOW architectures?
3. How does domain-specific Word2Vec compare to general-purpose GloVe?
4. Which emotions are easiest/hardest to classify?
5. How does embedding dimension (100d vs 300d) impact performance?
6. What's the effect of freezing vs fine-tuning pretrained embeddings?
7. Which architecture (Dense/CNN/LSTM) works best for emotion recognition?
8. How does class imbalance affect minority emotion classification?
9. What semantic clusters emerge in the embedding space?
10. Can embeddings capture emotion intensity and opposition?

Evaluation Rubric (100 points):

Criteria	Points	Description
Word2Vec Training	25	Correct implementation, thorough hyperparameter experiments, quality analysis
Emotion Classifier	30	Proper model architecture, effective training, good performance
Baseline Comparison	10	Fair comparison with TF-IDF baseline, insightful analysis
Embedding Analysis	20	Comprehensive visualization, analogy tasks, semantic exploration
Error Analysis	10	Thorough investigation of failures, actionable insights
Report Quality	5	Well-structured, clear explanations, professional presentation

Bonus Points (15):

- Implement attention mechanism with visualization (+5)
- Achieve >60% test accuracy on 27-class classification (+4)
- Train Word2Vec on 100M+ word corpus (+3)

- Implement contextualized embeddings (ELMo-style) (+3)

Target Metrics:

- **Word2Vec**: Spearman correlation >0.6 on similarity tasks
- **Emotion Classification**:
    - Minimum: 45% accuracy (27 classes)
    - Good: 52-58% accuracy
    - Excellent: >60% accuracy
- **F1-Score (macro)**: >0.50

****